

Draw It or Lose It — Memory vs. Storage Management

Overview. The Gaming Room plans to serve ~200 HD pictures (~8 MB each) while keeping the game responsive across Android, iOS, Windows, macOS, and the web. Below I separate short, practical guidance for memory (RAM) and storage (persistent media), then compare how each affects gameplay and the user experience.

Memory Management (RAM): how to render images fast without OOMs

Key idea: RAM is the game's kitchen—only what you're cooking right now should be on the counter. Keep full-resolution assets in the pantry (disk/cloud), and bring to RAM only down-scaled, display-sized versions just-in-time.

- **Decode to the display size, not full resolution.**

Always downsample to the pixels actually shown on screen (e.g., target view size or logical size) to shrink decode buffers dramatically. Android and Windows expose decode-pixel APIs to request smaller, subsampled bitmaps at load time; Apple recommends Image I/O for downsampling. This prevents wasting RAM on invisible pixels and reduces GC/alloc churn (*Android Developers, 2024; Microsoft, 2025; Apple, 2018*).

- **Use layered caches with strict budgets.**

Adopt a small in-memory LRU cache for the last few drawings (e.g., previous/current/next), paired with a disk cache for recently used files. Size RAM caches as a percent of total heap (e.g., 5–15%) and evict aggressively on low-memory signals. Libraries like Glide (Android) implement these policies well (*Android Developers, 2024*).

- **Prefetch one step ahead; stream on background threads.**

Start loading the next clue while the current one is on screen. Do all decoding I/O off the UI thread to avoid jank. Keep decode work chunked and cancelable to match the game's one-minute rounds and 30-second completion rule.

- **Choose memory-efficient pixel formats & texture compression.**

Prefer 24-bit or 16-bit formats when alpha isn't needed, and use platform texture compression where applicable (e.g., BC/Block Compression on Windows, ETC2/ASTC on mobile GPUs) to reduce GPU and system RAM footprints (*Microsoft, 2021*).

- **Release promptly & avoid hidden references.**

When a clue leaves the screen, drop strong references (e.g., set image sources to null) so GC can reclaim memory. Watch for accidental retention (long-lived singletons or static fields holding bitmaps) (*Microsoft, 2025*).

Storage Management (Persistent): how to size and organize the library

Key idea: Storage is the pantry. You can stock a large library here, but organize and compress assets so downloads are small and lookups are fast.

Capacity planning. Base images: $200 \times 8 \text{ MB} \approx 1,600 \text{ MB} (\approx 1.6 \text{ GB})$. Add ~10–20% for metadata, thumbnails, and cache = ~1.8–2.0 GB. Provision at least 3–5 GB on servers/object storage to allow growth, A/B variants, and future rounds.

- **Use modern image formats + derivatives.**

Store masters in a high-quality format (e.g., lossless PNG or high-quality JPEG), plus pre-generated derivatives: display-sized JPEG/WebP/AVIF renditions that match in-app decode sizes. Serving near-display-size assets lowers transfer time and the RAM needed after decode.

- **Adopt a CDN + immutable paths.**

Host the library on object storage (e.g., S3/GCS/Azure) behind a CDN. Use content-hashed, immutable filenames so clients can cache safely between rounds and app launches.

- **On-device storage policy.**

Keep only a small disk cache of recent clues (e.g., 50–100 images). Respect each platform's cache directories and purge on low space. Record lightweight metadata (IDs, tags, difficulty) in a compact database or JSON file.

- **Integrity, privacy, and updates.**

Checksum assets (e.g., SHA-256) when publishing; use HTTPS, signed URLs, and versioned manifests. Support background refresh so new image packs appear without app updates.

Comparison: how memory vs. storage affect gameplay

1. Purpose.

Memory (RAM) powers the moment-to-moment experience—decode, draw, and animate the current clue at 60 FPS without stutters. Storage (disk/cloud) holds the full catalog and supports fast retrieval and updates.

2. Constraints.

RAM is scarce and per-app limited; exceeding budgets causes garbage-collection spikes or OS kills. Storage is larger but slower; network adds latency. Therefore, we downsample and cache to fit RAM while fetching from storage just-in-time.

3. Typical tactics.

For RAM: decode-to-size, LRU memory cache, cancelable background loads, compressed textures. For storage: CDN-served derivatives, small on-device disk cache, content hashing, and manifest-based updates.

Bottom line: render only what you need at the size you need (RAM discipline), and store many versions smartly (storage discipline). These practices meet the client's requirement to display rapidly rendered images while supporting a growing, high-quality library.

References

Android Developers. (2024). Loading Large Bitmaps Efficiently.

<https://developer.android.com/topic/performance/graphics/load-bitmap>.

Android Developers. (2024). Handling bitmaps.

<https://developer.android.com/develop/ui/views/graphics>.

Apple. (2018). iOS Memory Deep Dive (WWDC18 Session 416).

<https://developer.apple.com/videos/play/wwdc2018/416/>.

Apple. (n.d.). Making changes to reduce memory use.

<https://developer.apple.com/documentation/xcode/making-changes-to-reduce-memory-use>.

Microsoft. (2025). Optimize animations, media, and images (UWP).

<https://learn.microsoft.com/en-us/windows/uwp/debug-test-perf/optimize-animations-and-media>

Microsoft. (2021). Block compression. [https://learn.microsoft.com/en-](https://learn.microsoft.com/en-us/windows/win32/direct2d/block-compression)

[us/windows/win32/direct2d/block-compression](https://learn.microsoft.com/en-us/windows/win32/direct2d/block-compression).